

USER_MANUAL

HelixCode User Manual

Version 2.0 | Last Updated: 2025-11-05

Table of Contents

1. [Introduction](#)
 2. [Installation & Setup](#)
 3. [LLM Providers](#)
 4. [Core Tools](#)
 5. [Workflows](#)
 6. [Advanced Features](#)
 7. [Development Modes](#)
 8. [API Reference](#)
 9. [Configuration](#)
 10. [Best Practices](#)
 11. [Troubleshooting](#)
 12. [FAQ](#)
-

1. Introduction

What is HelixCode?

HelixCode is an enterprise-grade distributed AI development platform that enables intelligent task division, work preservation, and cross-platform development workflows. It combines the power of 14+ AI providers with advanced tooling to accelerate your development process.

Key Capabilities

- **14+ AI Providers:** Anthropic Claude, Google Gemini, AWS Bedrock, Azure OpenAI, VertexAI, Groq, OpenAI, and more
- **Advanced Context:** Extended thinking, prompt caching, 2M token context windows
- **Smart Tools:** File operations, shell execution, browser control, web search, voice-to-code

- **Intelligent Workflows:** Plan mode, auto-commit, multi-file editing, context compression
- **Enterprise Features:** Checkpoint snapshots, 5-level autonomy modes, vision auto-switching
- **Distributed Architecture:** SSH-based worker pools with automatic management

Architecture Overview

HelixCode Server			
REST API	WebSocket	MCP Protocol	CLI/TUI
LLM Provider Layer (14+ providers)			
Tools: File, Shell, Browser, Web, Voice, Mapping			
Workflows: Plan, Commit, Edit, Compress, Confirm			
Advanced: Snapshots, Autonomy Modes, Vision Switch			
Database (PostgreSQL)		Cache (Redis)	

2. Installation & Setup

Prerequisites

- **Go 1.26+**
- **PostgreSQL 14+**
- **Redis 7+** (optional, recommended for production)
- **Git**
- **Docker & Docker Compose** (for containerized deployment)

Quick Start

```
# Clone repository
git clone https://github.com/HelixDevelopment/Helix-CLI.git
cd helixcode/HelixCode

# Install dependencies
go mod download
```

```
# Build
make build

# Run tests
make test

# Start development server
make dev
```

Production Setup

```
# Create database
createdb helixcode
createuser helixcode

# Set environment variables
export HELIX_DATABASE_PASSWORD=your_password
export HELIX_AUTH_JWT_SECRET=your_jwt_secret
export HELIX_REDIS_PASSWORD=your_redis_password

# Deploy with Docker Compose
docker-compose up -d

# Verify deployment
curl http://localhost/health
```

Initial Configuration

Create config/config.yaml:

```
server:
  address: "0.0.0.0"
  port: 8080

database:
  host: "localhost"
  port: 5432
  dbname: "helixcode"
  # Password via HELIX_DATABASE_PASSWORD env var

auth:
  token_expiry: 86400
  session_expiry: 604800
```

```
workers:
  health_check_interval: 30
  max_concurrent_tasks: 10

llm:
  default_provider: "anthropic"
  max_tokens: 4096
  temperature: 0.7
```

3. LLM Providers

3.1 Anthropic Claude

Most powerful coding assistant with extended thinking and prompt caching.

Models: - claude-4-sonnet / claude-4-opus (most capable) - claude-3-7-sonnet (enhanced reasoning) - claude-3-5-sonnet-latest (best for coding) - claude-3-5-haiku-latest (fast and efficient)

Setup:

```
export ANTHROPIC_API_KEY="sk-ant-your-key"
```

Advanced Features: - **Extended Thinking:** Automatic reasoning for complex problems - **Prompt Caching:** 90% cost reduction on repeated contexts -

Vision: Full image analysis support - **Tool Caching:** Cache tool definitions for multi-turn conversations

Example:

```
provider, _ :=
    anthropic.NewAnthropicProvider(ProviderConfigEntry{
        Type:    ProviderTypeAnthropic,
        APIKey: os.Getenv("ANTHROPIC_API_KEY"),
    })

request := &LLMRequest{
    Model:    "claude-4-sonnet",
    Messages: []Message{{
        Role: "user",
        Content: "Think step by step: design a microservices
            architecture",
    }},
    MaxTokens: 10000,
}
```

★

```
response, _ := provider.Generate(ctx, request)
```

3.2 Google Gemini

Massive 2M token context windows for entire codebase analysis.

Models: - gemini-2.5-pro (2M tokens, most capable) - gemini-2.5-flash (1M tokens, ultra-fast) - gemini-2.0-flash (fast multimodal)

Setup:

```
export GEMINI_API_KEY="your-gemini-key"
```

Key Features: - **2M Token Context:** Process entire codebases in one request - **Multimodal:** Text, images, and code understanding - **Flash Models:** Ultra-fast responses

Example:

★

```
helixcode llm provider set gemini --model gemini-2.5-pro
helixcode analyze --full-context --model gemini-2.5-pro
```

3.3 AWS Bedrock NEW

Enterprise AI platform with multiple model families.

Models: - Claude 4/3.x (via Bedrock) - Amazon Titan - AI21 Jurassic - Cohere Command

Setup:

```
# Configure AWS credentials
aws configure
```

```
# Or use IAM role
export AWS_REGION=us-east-1
```

Example:

```
provider, _ := bedrock.NewBedrockProvider(ProviderConfigEntry{
    Type: ProviderTypeBedrock,
    Region: "us-east-1",
})
```

3.4 Azure OpenAI NEW

Microsoft's enterprise OpenAI service with Entra ID authentication.

Setup:

```
export AZURE_OPENAI_ENDPOINT="https://your-  
resource.openai.azure.com/"  
export AZURE_OPENAI_API_KEY="your-key"
```

Features: - Entra ID (Azure AD) authentication - Deployment-based routing - Content filtering - Enterprise compliance

3.5 Google VertexAI NEW

Unified AI platform with Gemini and Claude Model Garden.

Setup:

```
gcloud auth application-default login  
export GOOGLE_CLOUD_PROJECT="your-project-id"
```

Access to: - Gemini 2.5 Pro/Flash - Claude via Model Garden - PaLM 2 models

3.6 Groq NEW

Ultra-fast inference with 500+ tokens/sec on LPU hardware.

Models: - llama-3.3-70b-versatile - mixtral-8x7b-32768

Setup:

```
export GROQ_API_KEY="your-groq-key"
```

Performance: First token < 100ms, 500+ tokens/sec sustained

3.7 Other Providers

OpenAI

```
export OPENAI_API_KEY="sk-your-key"
```

Models: GPT-4.1, GPT-4.5 Preview, GPT-4o, O1/O3/O4

Local (Ollama/Llama.cpp)

No API key required - 100% offline
ollama serve

Free Providers

- **XAI (Grok)**: No API key for basic usage
 - **OpenRouter**: Free models available
 - **GitHub Copilot**: Free with GitHub subscription
 - **Qwen**: 2,000 free requests/day
-

4. Core Tools

4.1 File System Tools NEW

Advanced file operations with intelligent caching and safe editing.

Capabilities: - Read single/multiple files - Write with atomic operations - In-place editing with search/replace - Pattern-based file search - Git-aware filtering - Permission checks

Example:

```
import "dev.helix.code/internal/tools/filesystem"

fs, _ := filesystem.NewFileSystemTools(&filesystem.Config{
    EnableCache: true,
    CacheSize:   1000,
})

// Read file
content, _ := fs.Read(ctx, "main.go")

// Edit with search/replace
fs.Edit(ctx, "main.go", &filesystem.EditOptions{
    Search: "oldFunc",
    Replace: "newFunc",
})

// Search files
results, _ := fs.Search(ctx, "TODO", &filesystem.SearchOptions{
    Pattern: "*.go",
})
```

★

```
Recursive: true,  
}))
```

4.2 Shell Execution NEW

Safe command execution with sandboxing and security controls.

Features: - Allowlist/blocklist security - Real-time output streaming - Timeout management - Working directory control - Dry-run mode

Example:

```
import "dev.helix.code/internal/tools/shell"  
  
executor, _ := shell.NewCommandExecutor(&shell.Config{  
    Allowlist: []string{"git", "npm", "go"},  
    Blocklist: []string{"rm", "dd", "mkfs"},  
    Timeout:   30 * time.Second,  
})  
  
result, _ := executor.Execute(ctx, &shell.Command{  
    Name: "git",  
    Args: []string{"status"},  
})
```

4.3 Browser Control NEW

Chrome automation for web scraping, testing, and interaction.

Capabilities: - Launch/attach to Chrome - Navigate and interact - Screenshots with annotation - Form filling - Console monitoring

Example:

```
import "dev.helix.code/internal/tools/browser"  
  
browser, _ := browser.NewBrowserController(&browser.Config{  
    Headless: true,  
})  
  
browser.Launch(ctx, &browser.LaunchOptions{})  
browser.Navigate(ctx, "https://example.com")  
screenshot, _ := browser.Screenshot(ctx,  
    &browser.ScreenshotOptions{  
        FullPage: true,  
    })
```


4.4 Codebase Mapping NEW

Tree-sitter AST parsing for deep code understanding.

Supported Languages: 30+ (Go, TypeScript, Python, Rust, Java, C++, etc.)

Example:

```
import "dev.helix.code/internal/tools/mapping"

mapper, _ := mapping.NewMapper(&mapping.Config{
    CacheDir: ".helix.cache",
})

codeMap, _ := mapper.MapCodebase(ctx, "/path/to/project",
    &mapping.MapOptions{
        Languages: []string{"go", "typescript"},
    })

for _, file := range codeMap.Files {
    fmt.Printf("%s: %d functions, %d classes\n",
        file.Path, len(file.Functions), len(file.Classes))
}
```

4.5 Web Tools NEW

Multi-provider search and HTML fetching/parsing.

Search Engines: - Google Custom Search - Bing Web Search - DuckDuckGo (no API key)

Example:

```
import "dev.helix.code/internal/tools/web"

wt, _ := web.NewWebTools(&web.Config{
    DefaultProvider: web.ProviderDuckDuckGo,
})

// Search
results, _ := wt.Search(ctx, "golang best practices",
    web.SearchOptions{
        MaxResults: 10,
    })
```

```

// Fetch and parse
markdown, metadata, _ := wt.FetchAndParse(ctx, "https://
    example.com")

```

4.6 Voice-to-Code NEW

Hands-free coding with Whisper transcription.

Example:

```

import "dev.helix.code/internal/tools/voice"

manager, _ := voice.NewVoiceInputManager(&voice.VoiceConfig{
    APIKey:      os.Getenv("OPENAI_API_KEY"),
    SampleRate:  16000,
    SilenceTimeout: 2 * time.Second,
})

// Record and transcribe in one operation
text, _ := manager.RecordAndTranscribe(ctx)

```

5. Workflows

5.1 Plan Mode NEW

Two-phase workflow: Plan → Act with interactive review.

Phases: 1. **Plan:** Analyze task, generate options, present choices 2. **Act:** Execute approved plan with progress tracking

Example:

```

import "dev.helix.code/internal/workflow/planmode"

planner, _ := planmode.NewPlanner(llmProvider)

// Generate plan
plan, _ := planner.GeneratePlan(ctx, &planmode.Task{
    Description: "Refactor authentication module",
    Context:     codebaseContext,
})

// Generate options
options, _ := planner.GenerateOptions(ctx, task)

```

```
// User selects option, then execute
executor.Execute(ctx, selectedOption)
```

CLI Usage:

```
helixcode plan "Refactor auth module"  
# Review plan  
helixcode execute --plan <plan-id>
```

5.2 Auto-Commit NEW

LLM-generated commit messages following conventions.

Features: - Semantic commit messages - Conventional commits (feat:, fix:, docs:, etc.) - Co-author attribution - Amend detection

Example:

```
import "dev.helix.code/internal/tools/git"  
  
coordinator, _ := git.NewAutoCommitCoordinator("/path/to/repo",  
    llmProvider)  
  
result, _ := coordinator.AutoCommit(ctx, git.CommitOptions{  
    Files: []string{"main.go"},  
    Author: git.Person{Name: "Dev", Email: "dev@example.com"},  
    Attributions: []git.Attribution{  
        git.GetClaudeAttribution(),  
    },  
})  
  
fmt.Printf("Committed: %s\n", result.Message)
```

CLI Usage:

```
git add .  
helixcode commit --auto
```

5.3 Multi-File Editing NEW

Atomic transactions across multiple files with automatic rollback.

Example:

```
import "dev.helix.code/internal/tools/multiedit"  
  
manager, _ := multiedit.NewTransactionManager()
```

```

tx, _ := manager.Begin()

tx.AddEdit(&multiedit.FileEdit{
    Path:    "file1.go",
    Content: newContent1,
})
tx.AddEdit(&multiedit.FileEdit{
    Path:    "file2.go",
    Content: newContent2,
})

// All or nothing
if err := manager.Commit(tx.ID); err != nil {
    manager.Rollback(tx.ID)
}

```

5.4 Context Compression NEW

Automatic conversation summarization for token optimization.

Strategies: - **Sliding Window:** Keep last N messages - **Semantic Summarization:** LLM-powered summaries - **Hybrid:** Best of both worlds (default)

Example:

```

import "dev.helix.code/internal/llm/compression"

coordinator := compression.NewCompressionCoordinator(llmProvider,
    compression.WithStrategy(compression.StrategyHybrid),
    compression.WithThreshold(180000), // 90% of 200K
)

if shouldCompress, _ :=
    coordinator.ShouldCompress(conversation); shouldCompress
{
    result, _ := coordinator.Compress(ctx, conversation)
    fmt.Printf("Saved %d tokens (%.1f%% reduction)\n",
        result.TokensSaved,
        float64(result.TokensSaved)/
        float64(result.Original.TokenCount)*100)
}

```

5.5 Tool Confirmation NEW

Interactive confirmation for dangerous operations.

Example:

```
import "dev.helix.code/internal/tools/confirmation"

coordinator := confirmation.NewConfirmationCoordinator()

result, _ := coordinator.Confirm(ctx,
    confirmation.ConfirmationRequest{
        ToolName: "bash",
        Operation: confirmation.Operation{
            Type:      confirmation.OpDelete,
            Description: "Delete temporary files",
            Target:      "/tmp/build-cache",
            Risk:        confirmation.RiskMedium,
        },
    },
)

if result.Allowed {
    // Execute operation
}
```

6. Advanced Features

6.1 Checkpoint Snapshots NEW

Git-based workspace snapshots with instant rollback.

Example:

```
import "dev.helix.code/internal/workflow/snapshots"

manager, _ := snapshots.NewManager("/path/to/repo")

// Create checkpoint
snapshot, _ := manager.CreateSnapshot(ctx,
    &snapshots.CreateOptions{
        Description: "Before refactoring",
        Tags:        []string{"stable"},
    },
)
```

```
// ... make changes ...

// Restore if needed
manager.RestoreSnapshot(ctx, snapshot.ID,
    &snapshots.RestoreOptions{
        CreateBackup: true,
    })
```

CLI Usage:

```
# Create snapshot
helixcode snapshot create "Before major refactor"
```

```
# List snapshots
helixcode snapshot list
```

```
# Restore
helixcode snapshot restore <snapshot-id>
```

6.2 Autonomy Modes NEW

5 levels of AI autonomy from manual to full automation.

Modes:

Level	Mode	Auto Context	Auto Apply	Confirmation	Best For
1	None		×	Always	Critical systems, auditing
2	Basic	■	×	Always	Fine-grained control
3	Basic Plus	■ Suggestions	■	Yes	Learning the tool
4	Semi Auto			Yes	Most workflows (default)
5	Full Auto			No	Trusted tasks, automation

Example:

```
import "dev.helixcode/internal/workflow/autonomy"

controller, _ := autonomy.NewAutonomyController(nil)
```

```
// Set mode
controller.SetMode(ctx, autonomy.ModeSemiAuto)

// Check permissions
action := autonomy.NewAction(
    autonomy.ActionApplyChange,
    "Apply code changes",
    autonomy.RiskMedium,
)

perm, _ := controller.RequestPermission(ctx, action)
if perm.Granted && !perm.RequiresConfirm {
    // Auto-approved
}
```

CLI Usage:

```
# Set autonomy mode
helixcode autonomy set semi-auto

# Get current mode
helixcode autonomy get

# Temporary escalation
helixcode autonomy escalate full-auto --duration 30m
```

6.3 Vision Auto-Switch NEW

Automatic switching to vision models when images detected.

Switch Modes: - **Once:** Just for this request - **Session:** For this conversation - **Persist:** Save as default

Example:

```
import "dev.helix.code/internal/llm/vision"

manager, _ := vision.NewVisionSwitchManager(&vision.Config{
    SwitchMode:    vision.SwitchSession,
    RequireConfirm: false,
}, registry)

result, _ := manager.ProcessInput(ctx, &vision.Input{
    Text: "What's in this image?",
    Files: []*vision.File{
```

```

        {Path: "screenshot.png", MIMEType: "image/png"},
    },
})

if result.SwitchPerformed {
    fmt.Printf("Switched to %s\n", result.ToModel.Name)
}

```

6.4 Cognee Memory Integration NEW

Persistent AI memory system with graph-based knowledge management.

Features: - Long-term conversation memory - Knowledge graph construction - Semantic search and retrieval - Cross-provider memory sharing - Automatic context management - Performance optimization

Configuration:

```

{
  "cognee": {
    "enabled": true,
    "mode": "local",
    "remote_api": {
      "service_endpoint": "https://api.cognee.ai",
      "api_key": "your-cognee-key"
    }
  },
  "providers": {
    "openai": {
      "cognee_enabled": true
    },
    "anthropic": {
      "cognee_enabled": true
    }
  }
}

```

CLI Usage:

```

# Enable Cognee for a provider
helix-config set providers.openai.cognee_enabled true

# Configure Cognee settings
helix-config set cognee.enabled true
helix-config set cognee.mode local

```



```
# Check Cognee status
helix-config get cognee
```

API Usage:

```
import "dev.helix.code/internal/memory"

// Initialize Cognee
cognee := memory.NewCogneeIntegration(&config.Cognee, logger)
err := cognee.Initialize(ctx, &config.Cognee)

// Store memory
memItem := memory.NewMemoryItem("id", "content", "type", 1.0,
    time.Now())
err = cognee.StoreMemory(ctx, memItem)

// Retrieve memory
query := memory.NewRetrievalQuery("search term", "type", 10)
results, err := cognee.RetrieveMemory(ctx, query)
```

7. Development Modes

HelixCode supports multiple development workflows:

Planning Mode

- Analyze requirements
- Create technical specifications
- Break down into tasks

Building Mode

- Code generation
- Dependency management
- Integration

Testing Mode

- Unit test generation
- Test execution
- Coverage analysis

Refactoring Mode

- Code analysis
- Optimization
- Restructuring

Debugging Mode

- Error analysis
- Root cause identification
- Fix generation

Deployment Mode

- Build packaging
 - Deployment automation
 - Rollback support
-

8. API Reference

REST API Endpoints

Authentication

- POST /api/auth/register - User registration
- POST /api/auth/login - Login
- POST /api/auth/logout - Logout
- POST /api/auth/refresh - Token refresh
- GET /api/auth/me - Current user

Tasks

- GET /api/tasks - List tasks
- POST /api/tasks - Create task
- GET /api/tasks/{id} - Get task
- PUT /api/tasks/{id} - Update task
- DELETE /api/tasks/{id} - Delete task

Workers

- GET /api/workers - List workers
- POST /api/workers - Register worker
- GET /api/workers/{id} - Get worker

- DELETE /api/workers/{id} - Remove worker

LLM Operations

- POST /api/llm/generate - Generate completion
 - POST /api/llm/stream - Stream completion
 - GET /api/llm/providers - List providers
 - GET /api/llm/models - List models
-

9. Configuration

Environment Variables

```
# Database
HELIX_DATABASE_HOST=localhost
HELIX_DATABASE_PORT=5432
HELIX_DATABASE_NAME=helixcode
HELIX_DATABASE_PASSWORD=secret

# Authentication
HELIX_AUTH_JWT_SECRET=your-secret
HELIX_AUTH_TOKEN_EXPIRY=86400

# Redis (optional)
HELIX_REDIS_HOST=localhost
HELIX_REDIS_PORT=6379
HELIX_REDIS_PASSWORD=secret

# AI Providers
ANTHROPIC_API_KEY=sk-ant-...
GEMINI_API_KEY=...
OPENAI_API_KEY=sk-...
GROQ_API_KEY=...
```

Configuration File

config/config.yaml:

```
server:
  address: "0.0.0.0"
  port: 8080
  read_timeout: 30s
  write_timeout: 30s
```

```
database:
  host: "${HELIX_DATABASE_HOST}"
  port: 5432
  dbname: "helixcode"
  max_connections: 100
  connection_timeout: 10s

llm:
  default_provider: "anthropic"
  providers:
    anthropic:
      enabled: true
      models: ["claude-4-sonnet", "claude-3-5-sonnet-latest"]
    gemini:
      enabled: true
      models: ["gemini-2.5-pro"]

workers:
  health_check_interval: 30s
  max_concurrent_tasks: 10
  task_timeout: 3600s

tools:
  filesystem:
    enable_cache: true
    cache_size: 1000
  shell:
    allowlist: ["git", "npm", "go", "make"]
  browser:
    headless: true
    timeout: 30s
```

10. Best Practices

Choosing Autonomy Mode

- **Start with Semi Auto:** Best for most workflows
- **Use Basic Plus:** When learning new features
- **Escalate to Full Auto:** For routine, trusted tasks
- **Drop to Basic/None:** For critical or risky operations

Managing Context

- Enable context compression for long conversations
- Use checkpoint snapshots before major changes
- Leverage codebase mapping for full project understanding
- Utilize Gemini 2.5 Pro for entire codebase analysis (2M tokens)

Security

- Always use tool confirmation for dangerous operations
- Review plans before execution in Plan Mode
- Set appropriate autonomy levels for your environment
- Use allowlists for shell execution
- Enable audit logging for compliance

Performance

- Use Groq for ultra-fast inference
 - Enable prompt caching with Claude
 - Use local models (Ollama) for offline work
 - Leverage file system caching
 - Configure appropriate timeouts
-

11. Troubleshooting

Common Issues

Problem: “No LLM provider available” **Solution:** Set provider API key and verify with `helixcode llm providers`

Problem: “Database connection failed” **Solution:** Check PostgreSQL is running and credentials are correct

Problem: “Worker health check failed” **Solution:** Verify SSH connectivity and worker installation

Problem: “Context window exceeded” **Solution:** Enable context compression or use larger context models (Gemini 2.5 Pro)

Problem: “Vision model not available” **Solution:** Ensure vision-capable provider is configured (Claude, Gemini, GPT-4o)

Error Messages

- `ErrProviderNotFound`: Configure provider in `config.yaml`
- `ErrModelNotSupported`: Check model availability with provider
- `ErrContextTooLarge`: Enable compression or reduce context
- `ErrPermissionDenied`: Review autonomy mode and permissions
- `ErrSnapshotNotFound`: Verify snapshot exists with `list` command

Debugging

Enable debug logging:

```
export HELIX_LOG_LEVEL=debug
helixcode server
```

Check logs:

```
tail -f /var/log/helix/helixcode.log
```

12. FAQ

Q: Which AI provider should I use? A: Claude 4 Sonnet for complex reasoning, Gemini 2.5 Pro for large codebases, Groq for speed.

Q: How much does it cost? A: HelixCode is open source. Provider costs vary (free options: XAI Grok, OpenRouter, Copilot).

Q: Can I run it offline? A: Yes, use Ollama or Llama.cpp local providers.

Q: What's the difference between modes? A: Semi Auto is best for most work (auto context, manual approval). Full Auto is for trusted automation.

Q: How do I backup my work? A: Use checkpoint snapshots or configure automatic checkpoints.

Q: Can multiple users collaborate? A: Yes, via distributed workers and shared sessions.

Q: What languages are supported? A: 30+ via codebase mapping: Go, TypeScript, Python, Rust, Java, C++, etc.

Q: How do I switch between providers? A: `helixcode llm provider set <provider> --model <model>`

Q: Is my code sent to AI providers? A: Only what you explicitly request. Use local models for full privacy.

Q: How do I contribute? A: See CONTRIBUTING.md in the repository.

Appendix A: Command Reference

Server

helixcode server	# Start server
helixcode health	# Health check

LLM

helixcode llm providers	# List providers
helixcode llm models	# List models
helixcode llm provider set <name>	# Set provider
helixcode llm generate "prompt"	# Generate

Plan Mode

helixcode plan "task description"	# Create plan
helixcode plan list	# List plans
helixcode plan execute <id>	# Execute plan

Snapshots

helixcode snapshot create "desc"	# Create snapshot
helixcode snapshot list	# List snapshots
helixcode snapshot restore <id>	# Restore snapshot
helixcode snapshot compare <a> 	# Compare snapshots

Autonomy

helixcode autonomy get	# Get current mode
helixcode autonomy set <mode>	# Set mode
helixcode autonomy escalate <mode>	# Temporary escalation

Workers

helixcode worker add <host>	# Add worker
helixcode worker list	# List workers
helixcode worker remove <id>	# Remove worker

Tasks

helixcode task create "desc"	# Create task
helixcode task list	# List tasks
helixcode task status <id>	# Task status

Appendix B: Keyboard Shortcuts (TUI)

Key	Action
Ctrl+C	Exit
Tab	Next panel
Shift+Tab	Previous panel
Enter	Execute/Confirm
Esc	Cancel
↑/↓	Navigate list
PgUp/PgDn	Scroll page
Home/End	First/last item
Ctrl+R	Refresh
Ctrl+P	Plan mode
Ctrl+S	Create snapshot

Appendix C: Configuration Examples

High-Performance Setup

```
llm:
  default_provider: "groq"

tools:
  filesystem:
    enable_cache: true
    cache_size: 5000

workers:
  max_concurrent_tasks: 20
```

Enterprise Security Setup

```
autonomy:
  default_mode: "basic"

tools:
  confirmation:
    enabled: true
```



```
    audit_log: "/var/log/helix/audit.log"
  shell:
    allowlist: ["git"]
    blocklist: ["rm", "dd", "mkfs", "sudo"]
```

Large Codebase Setup

```
llm:
  default_provider: "gemini"
  max_tokens: 2000000

tools:
  mapping:
    enable_cache: true
    languages: ["go", "typescript", "python"]
```

For more information: - Documentation: <https://docs.helixcode.dev> - GitHub: <https://github.com/HelixDevelopment/Helix-CLI> - Community: <https://community.helixcode.dev> - Issues: <https://github.com/HelixDevelopment/Helix-CLI/issues>

Version: 2.0 **Last Updated:** 2025-11-05 **License:** See LICENSE file